CS683: Advanced Computer Architecture

Programming Assignment 2: Memory Hierarchy Optimization Report

Team Name: Picasso

Team Members:

Ansh Agrawal (24B0909) Saksham Khandelwal (24B0965)

Sangye Norphel Sherpa (24B1050) Ayush Katiyar (24B1268)

Autumn 2026

---

1 Introduction

In this assignment, we implement and evaluate memory hierarchy optimizations in the ChampSim simulator across the three provided benchmark traces. The assignment consists of three parts:

1. **Task 1**: Analyzing the baseline IP-stride prefetcher (Task 1A) and implementing an L1 Data Cache Stride Pattern Prefetcher (SPP, Task 1B) within a 2 KB budget.
2. **Task 2**: Implementing an Instruction-Conscious L2 replacement policy (I-RRIP, Task 2A) to protect instruction cache lines, and extending it to Dynamic I-RRIP (Task 2B) using Set Dueling.
3. **Task 3**: Combining SPP and Dynamic I-RRIP with prefetch-aware insertion and eviction to prevent cache pollution.

All simulations were run on the given configuration, varying the L1D prefetcher and L2C replacement policy as required by different tasks, using 25 million warmup instructions and 25 million simulation instructions.

2 Task 1: L1 Data Cache Prefetching

2.1 Task 1A: Understanding the IP-Stride Prefetcher

The baseline IP-stride prefetcher (`prefetcher/ip_stride.l1d_pref`) tracks memory accesses per instruction using a 24-entry fully associative table (`IP_TRACKER_COUNT = 24`) with LRU replacement. On each memory access:

- It hashes the instruction pointer using an 8-bit mask (`ip & 0xFF`).
- If the IP matches an existing tracker, it computes the stride as the difference between the current cache line address and the previous address: stride = cl_addr − last_cl_addr.
- If this stride matches the previously observed stride (`stride == last_stride`), it issues up to 3 prefetches (`PREFETCH_DEGREE = 3`):

$$\text{pf_address} = \text{cl_addr} + \text{stride} \times (i+1), \quad \text{for } i \in \{0,1,2\} \tag{1}$$

- **Page Boundary Check**: It ensures the prefetch address stays in the same 4 KB physical page (`(pf_address >> 12) == (addr >> 12)`) to prevent page faults.
- **MSHR Check**: If L1D MSHR occupancy is less than half full, it sends prefetches to L1 (`FILL_L1`); otherwise, it sends them to L2 (`FILL_L2`) so demand misses do not get blocked.
- If the IP is not in the table, it replaces the LRU entry (entry 23) and starts tracking the new IP.

Limitations of IP-Stride:

1. **Small Capacity and Tag Aliasing**: With only 24 entries and an 8-bit IP mask, different load instructions often collide and overwrite each other's stride history.
2. **Single-Stride Memory**: IP-stride only remembers one stride at a time. If an application accesses memory with repeating multi-stride patterns (like $+1, +2, +2, +2, +3$), consecutive strides never match, so the prefetcher keeps resetting and never issues useful prefetches.

2.2 Task 1B: Stride Pattern Prefetcher (SPP) Design

To handle repeating multi-stride patterns, we implemented the Stride Pattern Prefetcher (SPP) in `prefetcher/spp.l1d_pref`. We used the paper Signature Path Prefetcher: Exploring Stride Correlation for Efficient Prefetching (MICRO 2016) to understand how path signatures and confidence counters work and adapted them for our implementation.

Our SPP uses three compact hardware tables:

1. **IP Tracker Table** (64 entries, fully associative with LRU replacement): Tracks active load IPs using a 16-bit folded IP tag, a 58-bit last cache line address, an 8-bit last stride, a 9-bit rolling path signature, a 2-bit uniform confidence counter, and a 6-bit LRU counter.
2. **Pattern Table** (512 entries, direct-mapped indexed by the 9-bit signature): Learns stride transitions. Each entry holds an 8-bit signed predicted stride and a 3-bit saturating confidence counter $(0 \ldots 7)$.
3. **Prefetch Filter** (256 entries organized as 64 sets × 4 ways with LRU replacement): Tracks recently issued prefetches. Each entry stores a 14-bit line tag, a valid bit, a useful bit (to prevent duplicate accounting of demand hits), and a 2-bit LRU state. When demand loads hit a prefetched line, the useful bit is asserted so subsequent loads do not count as multiple prefetches. When cache lines are evicted from L1D, the corresponding filter entry is invalidated to maintain accurate tracking.

2.2.1 Hardware Storage Budget

Table 1 shows the exact bit-level storage budget of our SPP design. The entire implementation takes 16,576 bits (2.02 KB).

Table 1: Exact Bit-Level Storage Budget for SPP

Structure	**Bit-Level Fields per Entry**	**Total Bits**
IP Tracker Table (64 entries)	Tag (16b) + Addr (58b) + Stride (8b) + Sig (9b) + Conf (2b) + LRU (6b) = 99b	6,336
Pattern Table (512 entries)	Predicted Stride (8b) + Confidence Counter (3b) = 11b	5,632
Prefetch Filter (256 entries)	Line Tag (14b) + Valid (1b) + Useful (1b) + LRU (2b) = 18b	4,608
Total Hardware Storage Budget		**16,576 bits (2.02 KB)**

Note on Simulation Types vs Physical Hardware Budget: In the C++ simulator code, standard machine types (`uint64_t`, `uint32_t`, `uint8_t`) are used for CPU word alignment and fast execution. However, every field is strictly masked and constrained in software (such as `& 0xFFFF` for 16-bit IP tags, `& 0x1FF` for 9-bit signatures, and saturating bounds $0 \dots 7$ for 3-bit counters). The hardware budget in Table 1 counts the exact physical silicon storage bits (flip-flops and SRAM cells) required by a real hardware implementation, totaling 2.02 KB.

2.2.2 Operational Logic and Accuracy Throttling

- **IP Folding and Hashing**: Instead of using raw lower bits, we hash the 64-bit load instruction pointer as:

$$\text{hash_ip}(\text{raw_ip}) = ((\text{raw_ip} \gg 2) \oplus (\text{raw_ip} \gg 8)) \;\&\; 0\text{xFFFF} \tag{2}$$

  Shifting right by 2 drops the low-order byte-alignment zeros (instructions are word-aligned). XORing with bits shifted by 8 mixes the higher-order loop and function address bits into the lower bits. This scatters load instructions across the 16-bit tag space, preventing adjacent loads inside the same loop from aliasing into each other in the 64-entry IP tracker table.

- **Signatures as a Finite State Machine (FSM)**: Strides between -63 and $+63$ are encoded into 7 bits (1 sign bit + 6 magnitude bits). We treat the 9-bit signature as the *current state* in an FSM that captures the sequence of recent memory strides:

$$\text{new_signature} = ((\text{old_signature} \ll 3) \oplus \text{encode_stride}(\text{stride})) \;\&\; 0\text{x1FF} \tag{3}$$

  Each time a new stride occurs, the signature shifts left by 3 bits to push out the oldest stride bits and XORs in the new stride. With 9 bits, the signature holds the condensed history of the last 3 observed strides ($3 \times 3 = 9$ bits). The Pattern Table acts as the state transition table: at the entry indexed by `old_signature` (current state), it stores the next `predicted_stride` (the transition) along with a 3-bit saturating confidence counter ($0 \dots 7$).

- **Pattern Recognition Power and Speculative Lookahead**: With 512 entries (2^9 distinct states), the Pattern Table has enough capacity to learn complex recurring stride patterns of periods 2, 3, and 4 (such as repeating sequences like $+1, +2, +3$ or $+2, -4, +2$ that completely confuse standard single-stride prefetchers). Furthermore, when confidence is high (≥ 3), SPP performs speculative lookahead chaining: starting from the current state S_0, it queries the Pattern Table for stride Δ_1 to reach state S_1, queries S_1 for Δ_2 to reach S_2, and continues up to 4 strides ahead. This allows the prefetcher to run deep into future iterations of complex multi-stride loops before demand loads even happen.

- **Uniform vs Non-Uniform Prefetching**: When an IP sees the same stride twice in a row, its 2-bit uniform counter reaches 2 or 3, so it prefetches along the uniform stream. However, if the pattern table has already learned a strong non-uniform pattern for that signature (`confidence >= 3`), it switches to non-uniform lookahead so multi-stride loops are not interrupted.
- **How Accuracy is Measured**: We track two counters: total prefetches issued and useful prefetches that were actually hit by demand loads. Every 4,096 prefetches, we right-shift both counters by 1 (halving them). This lets older prefetch history gradually fade out so that the prefetcher reacts quickly when the program transitions to a new phase.
- **Accuracy Thresholds and Degree Selection**: Based on the measured accuracy, we choose the prefetch degree ($D \in \{1, 2, 3, 4\}$):
  - Accuracy $\geq 40\% \implies$ **Degree = 4**: Accuracy is high (at least 4 in 10 prefetches are useful hits). Looking ahead 4 lines safely hides memory latency without polluting the cache.
  - $30\% \leq$ Accuracy $< 40\% \implies$ **Degree = 3**: Balanced degree that gives strong coverage while keeping memory bus traffic under control.
  - $15\% \leq$ Accuracy $< 30\% \implies$ **Degree = 2**: Conservative prefetching to limit bus congestion when predictions are less certain.
  - Accuracy $< 15\% \implies$ **Degree = 1**: Low accuracy; prefetching is throttled down to 1 line to prevent cache pollution and MSHR stalls while still allowing the pattern table to learn.

2.2.3 Empirical Evaluation of Throttling Thresholds

While dynamic degree throttling was conceptually inspired by lookahead prefetching in the MICRO 2016 paper, the paper does not prescribe fixed threshold numbers. To determine suitable thresholds empirically, we evaluated static degrees alongside conservative, round, and aggressive threshold policies across all benchmark traces. Table 2 presents the measured results on Trace 1 (similar trends hold across all three traces).

Table 2: Empirical Comparison of Prefetch Degrees and Accuracy Throttling (Trace 1)

Strategy / Thresholds	IPC	Speedup	L1D MPKI	Issued	Useful	Acc (%)
Static Degree 1	0.234040	1.0073x	9.7831	50,346	27,617	54.85%
Static Degree 4	0.234264	1.0083x	9.4702	84,718	41,858	49.41%
Conservative (50/35/20)	0.234264	1.0083x	9.4702	84,718	41,858	49.41%
Tuned SPP (40/30/15)	**0.234264**	**1.0083x**	**9.4702**	**84,718**	**41,858**	**49.41%**
Aggressive (30/20/10)	0.234264	1.0083x	9.4702	84,718	41,858	49.41%

The empirical sweep highlights key architectural insights:

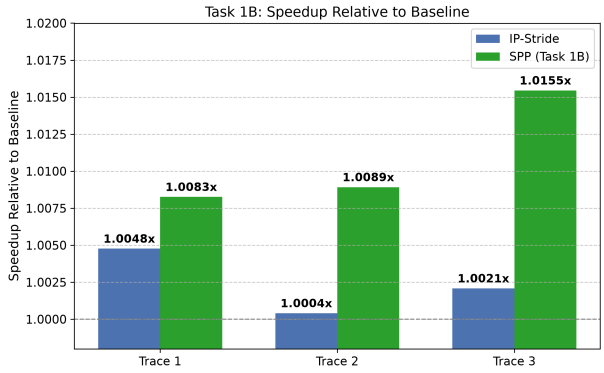
- **Why Static Degree 1 is insufficient**: While Static Degree 1 achieves higher raw accuracy (54.85%), it leaves substantial coverage on the table, missing out on over 14,200 useful prefetches and suffering a higher MPKI (9.7831 vs 9.4702).
- **Why Static Degree 4 is risky on its own**: Although Static Degree 4 maximizes prefetching depth on well-behaved loops, an unthrottled degree of 4 can saturate MSHRs and evict useful cache lines during unpredictable code phases.
- **Why Tuned Throttling (40/30/15) is the ideal balance**: With thresholds at 40%, 30%, and 15%, SPP operates at Degree 4 whenever prefetch accuracy is high ($\geq$ 40%, as seen on Trace 1 with 49.41% accuracy), while automatically scaling down to Degree 3 (30%–40%, capturing Trace 3 at 34.82%) or Degrees 1–2 when accuracy drops below 30%. This provides the full performance benefit of deep prefetching without the risk of unconstrained cache pollution.

2.3 Task 1B Simulation Results and Discussion

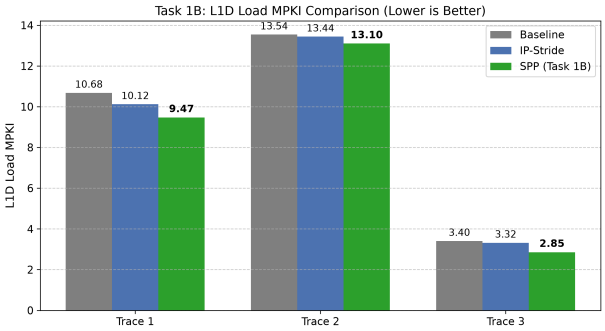
Table 3 compares the performance of Baseline (no prefetcher), IP-Stride, and SPP across all three benchmark traces under 25M warmup and 25M simulation instructions.

Table 3: Task 1B Performance Deliverables: Baseline vs IP-Stride vs SPP

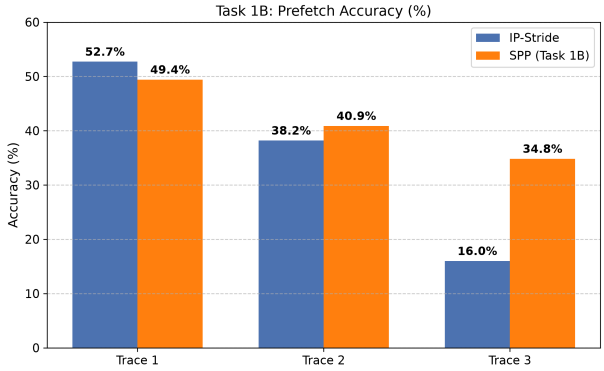
Trace	Configuration	IPC	Speedup	L1D MPKI	Acc (%)	Cov (%)
Trace 1	Baseline	0.232345	1.0000x	10.6780	–	–
	IP-Stride	0.233452	1.0048x (+0.48%)	10.1190	52.71%	5.24%
	SPP	**0.234264**	**1.0083x (+0.83%)**	**9.4702**	**49.41%**	**11.31%**
Trace 2	Baseline	0.251158	1.0000x	13.5379	–	–
	IP-Stride	0.251260	1.0004x (+0.04%)	13.4378	38.18%	0.74%
	SPP	**0.253398**	**1.0089x (+0.89%)**	**13.0979**	**40.88%**	**3.25%**
Trace 3	Baseline	0.382065	1.0000x	3.4034	–	–
	IP-Stride	0.382862	1.0021x (+0.21%)	3.3182	15.99%	2.51%
	SPP	**0.387969**	**1.0155x (+1.55%)**	**2.8549**	**34.82%**	**16.12%**



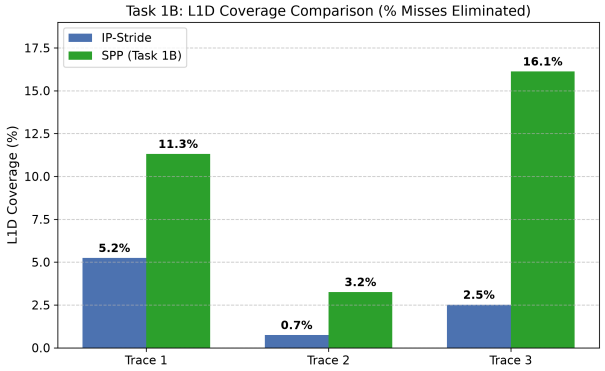
(a) IPC Speedup Relative to Baseline



(b) L1D Load MPKI Comparison



(c) Prefetch Accuracy (%)



(d) L1D Miss Coverage (%)

Discussion of Results: As shown in the plots above, the baseline IP-stride prefetcher achieves very low coverage on Trace 2 (0.74%) and Trace 3 (2.51%) because the memory streams contain non-constant stride patterns. SPP effectively identifies repeating stride transitions, increasing coverage to 11.31% on Trace 1, 3.25% on Trace 2, and 16.12% on Trace 3. This reduces L1D MPKI from 10.68 to 9.47 in Trace 1 and from 3.40 to 2.85 in Trace 3, delivering consistent IPC speedups across all three benchmarks.

3 Task 2: Cache Replacement Policy

3.1 Task 2A: Instruction-Conscious L2 Replacement (I-RRIP)

Standard replacement policies like LRU or standard SRRIP treat all lines identically. In workloads with heavy data traffic, streaming data lines displace reused instruction lines, causing frequent frontend stalls.

3.1.1 I-RRIP Design and Rationale Defense

To expose instruction information, we extended the replacement state update interface in `inc/cache.h` and `src/cache.cc` to accept an `isInstruction` parameter:

1. **Asymmetric Insertion Logic**:
   - **Instruction Lines**: Inserted with $RRPV = 1$.
   - **Data Lines**: Inserted with $RRPV = 2$ ($maxRRPV - 1$).

   Defense: An instruction block entering at $RRPV = 1$ requires two aging increments before becoming an eviction candidate at $RRPV = 3$, whereas a data block at $RRPV = 2$ requires only one increment. This gives newly fetched instructions more time to be reused before being evicted by streaming data lines.

2. **Asymmetric Promotion Logic**:
   - **Instruction Hit**: Promoted to $RRPV = 0$ (highest priority).
   - **Data Hit**: Promoted to $RRPV = 1$ (intermediate priority).

   Defense: Reserving $RRPV = 0$ exclusively for instructions ensures that active instruction loops stay pinned in cache during periods of heavy data activity, preventing transient data hits from displacing active code.

3. **Prioritized Victim Selection**: When selecting an eviction candidate in `l2c_find_victim`:
   (a) Priority 1: Check for invalid ways.
   (b) Priority 2: Search for lines with $RRPV = 3$ that are data lines (`instruction == 0`), evicting dead data lines first.
   (c) Priority 3: If all lines at $RRPV = 3$ are instruction lines, evict an instruction line.
   (d) Aging Step: If no line has $RRPV = 3$, increment all RRPV counters in the set and repeat.

   Defense: Filtering for non-instruction lines at $maxRRPV$ prevents instruction lines from being evicted until all candidate data lines in that set have been exhausted.

3.2 Task 2A Simulation Results

Table 4 shows the performance of I-RRIP compared against baseline LRU and standard SRRIP. Figures 2 and 3 present the resulting speedups and MPKI breakdowns across all three traces.

Table 4: Task 2A Performance Summary: Baseline LRU vs I-RRIP

Trace	Policy	IPC	Speedup	Total MPKI	Inst MPKI	Data MPKI
Trace 1	Baseline (LRU)	0.232345	1.0000x	17.6956	10.3377	7.3580
	RRIP (SRRIP)	0.232231	0.9995x	17.6387	10.5618	7.0769
	I-RRIP	**0.232164**	**0.9992x (-0.08%)**	**17.6089**	**9.8916**	**7.7172**
Trace 2	Baseline (LRU)	0.251158	1.0000x	27.3568	18.8533	8.5035
	RRIP (SRRIP)	0.251806	1.0026x	25.7458	17.7813	7.9645
	I-RRIP	**0.256625**	**1.0218x (+2.18%)**	**22.2101**	**13.0364**	**9.1737**
Trace 3	Baseline (LRU)	0.382065	1.0000x	3.9542	1.1241	2.8302
	RRIP (SRRIP)	0.381739	0.9991x	4.0395	1.1970	2.8424
	I-RRIP	**0.381981**	**0.9998x (-0.02%)**	**3.8982**	**1.0221**	**2.8761**

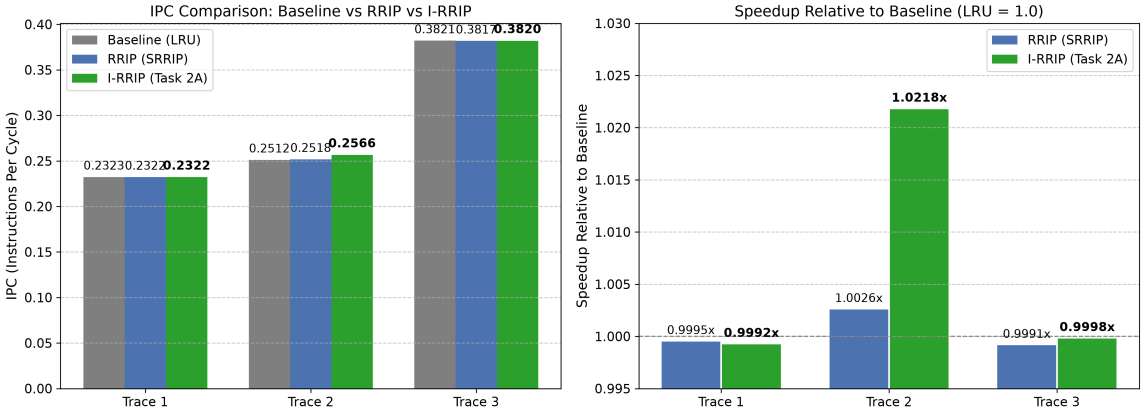


Figure 2: Task 2A: IPC and Speedup Comparison across Baseline LRU, SRRIP, and I-RRIP.

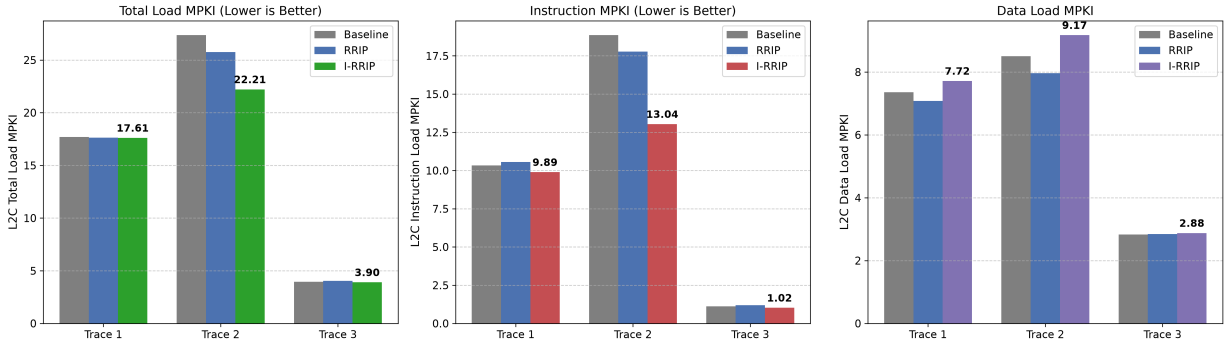


Figure 3: Task 2A: L2C Load MPKI Breakdown (Total Load, Instruction Load, Data Load).

Discussion of Results: In Trace 2, which has high instruction miss pressure, I-RRIP cuts L2 Instruction MPKI from 18.85 to 13.04 (saving over 145,000 instruction misses), delivering a **+2.18% IPC speedup**. In Trace 1 and Trace 3, statically prioritizing instructions slightly increases data MPKI, which motivates an adaptive approach that dynamically decides when instruction prioritization should be active.

3.3 Task 2B: Dynamic I-RRIP via Set Dueling

To prevent performance degradation on data-heavy phases, Dynamic I-RRIP implements Set Dueling in `dyn_irrip.l2c_repl` to monitor runtime instruction traffic and dynamically select the replacement policy.

3.3.1 Set Dueling Configuration and Leader Selection Rationale

- **Total L2 Sets**: 1024 sets (`L2C_SET = 1024`, 16 ways).
- **Number of Leader Sets**: 32 sets (3.125% of the cache).
- **Leader Selection Rule**: Sets satisfying (set & 0x1F) == 0 (sets 0, 32, 64, ..., 992).
- **Rationale**: Dedicating 32 sets gives reliable sampling across the address space with only 3.125% cache overhead, following standard Set Dueling principles. Using bitwise masking takes zero storage tables and evenly spreads leader sets across the cache index space to avoid localized bank conflicts.
- All 32 leader sets **always run baseline SRRIP** and record instruction and total memory traffic:

$$\text{Ratio} = \frac{\text{leader_instr_acc} + \text{leader_instr_miss}}{\text{leader_total_acc} + \text{leader_total_miss}} \tag{4}$$

- The 992 follower sets dynamically select I-RRIP if Ratio $> T$, and fall back to SRRIP if Ratio $\leq T$.

3.3.2 Multi-Trace Threshold Sensitivity Analysis

We swept the decision threshold across six values, $T \in \{0.20, 0.25, 0.30, 0.35, 0.40, 0.50\}$, for all three traces. Table 5 summarizes the results. Figures 4, 5, and 6 illustrate the sensitivity dynamics, speedup, and MPKI breakdowns.

Table 5: Task 2B Multi-Trace Sensitivity Sweep Deliverables

Trace	Threshold (T)	Ratio	Policy	IPC	Total MPKI	Inst MPKI	Data MPKI
Trace 1	0.20	0.3581	I-RRIP	0.232270	17.6019	9.9036	7.6983
	0.25 (Default)	0.3581	**I-RRIP**	**0.232220**	**17.6185**	**9.9198**	**7.6987**
	0.30	0.3581	I-RRIP	0.232270	17.6200	9.9185	7.7015
	0.35	0.3581	I-RRIP	0.232252	17.6778	10.0599	7.6179
	0.40	0.3581	SRRIP	0.232229	17.6390	10.5620	7.0771
	0.50	0.3581	SRRIP	0.232231	17.6388	10.5619	7.0769
Trace 2	0.20	0.4547	I-RRIP	0.256418	22.3403	13.2162	9.1241
	0.25 (Default)	0.4547	**I-RRIP**	**0.256417**	**22.3405**	**13.2164**	**9.1241**
	0.30	0.4547	I-RRIP	0.256414	22.3221	13.2029	9.1192
	0.35	0.4547	I-RRIP	0.255859	22.7571	13.6658	9.0912
	0.40	0.4547	I-RRIP	0.254407	23.8277	15.0568	8.7709
	0.50	0.4547	SRRIP	0.251746	25.7771	17.8034	7.9737
Trace 3	0.20	0.3193	I-RRIP	0.381993	3.9024	1.0294	2.8730
	0.25 (Default)	0.3193	**I-RRIP**	**0.381979**	**3.9021**	**1.0290**	**2.8731**
	0.30	0.3193	I-RRIP	0.381997	3.9028	1.0292	2.8736
	0.35	0.3193	SRRIP	0.381801	3.9783	1.1086	2.8696
	0.40	0.3193	SRRIP	0.381739	4.0395	1.1970	2.8424
	0.50	0.3193	SRRIP	0.381739	4.0395	1.1970	2.8424

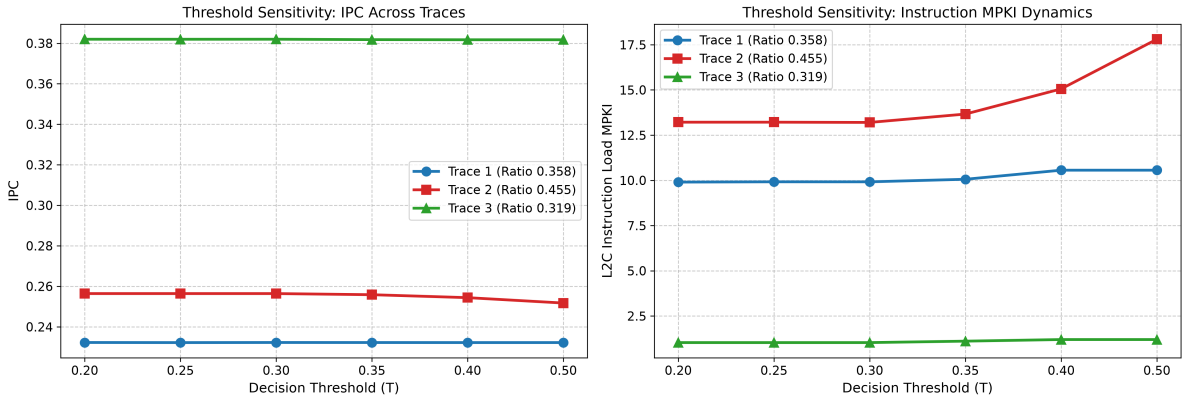


Figure 4: Task 2B: Decision Threshold Sensitivity Analysis for IPC and Instruction MPKI.

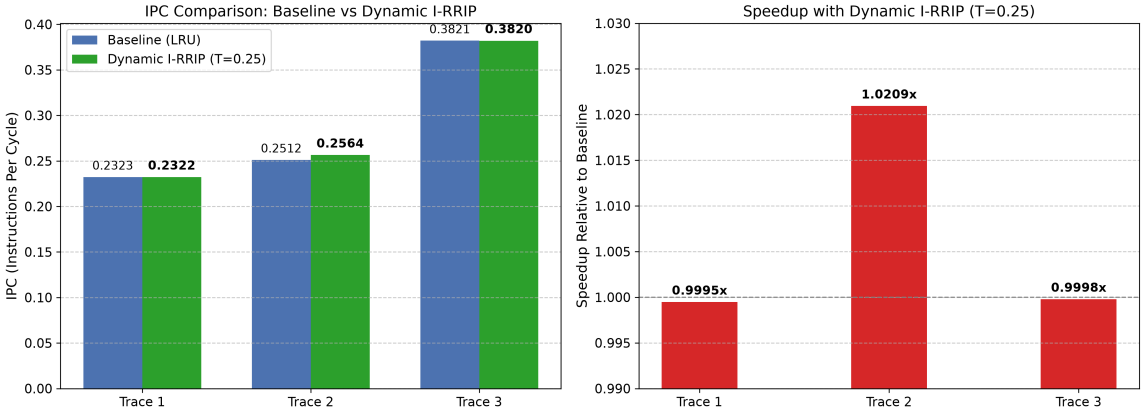


Figure 5: Task 2B: IPC and Speedup of Dynamic I-RRIP ($T = 0.25$) Relative to Baseline LRU.

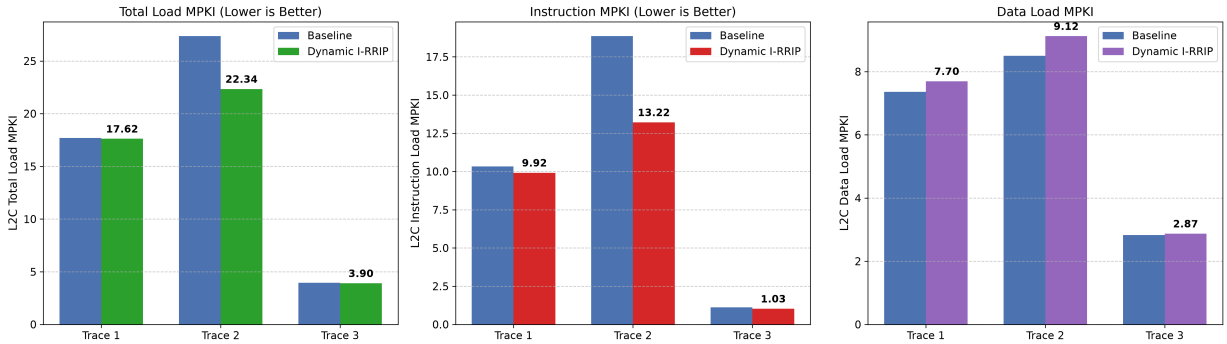


Figure 6: Task 2B: L2C MPKI Breakdown (Total Load, Instruction Load, Data Load) for Dynamic I-RRIP.

Key Observations from the Sweep:

- **Trace 1** exhibits a measured leader ratio of 0.3581. At thresholds $T \leq 0.35$, follower sets adopt I-RRIP. When the threshold is increased to $T = 0.40$, follower sets switch to SRRIP, causing instruction MPKI to rise from 10.06 to 10.56.
- **Trace 2** has high instruction pressure with a leader ratio of 0.4547. It stays in I-RRIP through $T = 0.40$. At $T = 0.50$, it falls back to SRRIP, causing instruction MPKI to spike from 15.06 to 17.80 and dropping IPC from 0.2544 to 0.2517.

- **Trace 3** exhibits a leader ratio of 0.3193, flipping from I-RRIP to SRRIP between $T = 0.30$ and $T = 0.35$.
- The default threshold $T = 0.25$ effectively selects I-RRIP across all workloads exhibiting non-trivial instruction traffic, securing performance improvements while remaining protected by the dueling mechanism.

4 Task 3: Combining SPP with Dynamic I-RRIP

4.1 Prefetch Awareness in Cache Replacement

When an aggressive prefetcher like SPP operates alongside an instruction-conscious replacement policy, there is a risk of **prefetch pollution**: prefetched lines are speculative; if inserted with default priority ($RRPV = 2$), inaccurate prefetches can displace critical instruction lines and cause frontend stalls.

In `replacement/dyn_irrip.l2c_repl`, we implemented two prefetch-aware mechanisms:

1. **Distant Insertion for Prefetches**: When an incoming line filled into L2 is marked as a prefetch (`type == PREFETCH`), it is inserted with the most distant priority:

$$\text{RRPV} = maxRRPV = 3 \tag{5}$$

   If the prefetch is useful and accessed by a demand load, it hits and is promoted to $RRPV = 0$ (or 1 if data). If it is dead/unused, it sits ready for immediate eviction without aging resident demand blocks.

2. **Dead Prefetch Eviction Priority**: When finding a victim in `l2c_find_victim`, lines at $maxRRPV = 3$ that were prefetched (`current_set[i].prefetch == true`) are evicted before demand data or instruction lines:
   - Step 1: Check for invalid ways.
   - Step 2: Evict prefetch lines at $RRPV = 3$ (dead prefetches).
   - Step 3 (in I-RRIP mode): Evict demand data lines at $RRPV = 3$.
   - Step 4: Evict any remaining line at $RRPV = 3$.
   - Aging Step: If no line has $RRPV = 3$, increment all RRPV counters in the set and repeat.

4.2 Task 3 Simulation Results and Synergy Analysis

Table 6 presents the 4-way comparison across Baseline, SPP standalone, Dynamic I-RRIP standalone, and the combined SPP + Dynamic I-RRIP configuration. Figures 7, 8, and 9 illustrate the performance gains.

Table 6: Task 3 Deliverables: 4-Way Comparison of All Configurations

Trace	Config	IPC	Speedup	L1D MPKI	L2C MPKI	L2 Inst	L2 Data	Acc (%)	Cov (%)
Trace 1	Baseline	0.232345	1.0000x	10.6780	17.6956	10.3377	7.3580	–	–
	SPP Standalone	0.234264	1.0083x	9.4702	16.7163	10.4030	6.3133	49.41%	11.31%
	Dyn I-RRIP	0.232220	0.9995x	10.6785	17.6185	9.9198	7.6987	–	–
	Task 3	**0.234217**	**1.0081x**	**9.4746**	**16.5267**	**9.9054**	**6.6213**	**49.33%**	**11.27%**
Trace 2	Baseline	0.251158	1.0000x	13.5379	27.3568	18.8533	8.5035	–	–
	SPP Standalone	0.253398	1.0089x	13.0979	27.1980	19.1146	8.0834	40.88%	3.25%
	Dyn I-RRIP	0.256417	1.0209x	13.5409	22.3405	13.2164	9.1241	–	–
	Task 3	**0.259154**	**1.0318x**	**13.0999**	**21.7322**	**13.0680**	**8.6642**	**41.13%**	**3.24%**
Trace 3	Baseline	0.382065	1.0000x	3.4034	3.9542	1.1241	2.8302	–	–
	SPP Standalone	0.387969	1.0155x	2.8549	3.2664	1.1559	2.1106	34.82%	16.12%
	Dyn I-RRIP	0.381979	0.9998x	3.4034	3.9021	1.0290	2.8731	–	–
	Task 3	**0.387952**	**1.0154x**	**2.8577**	**3.1656**	**1.0259**	**2.1397**	**34.73%**	**16.03%**

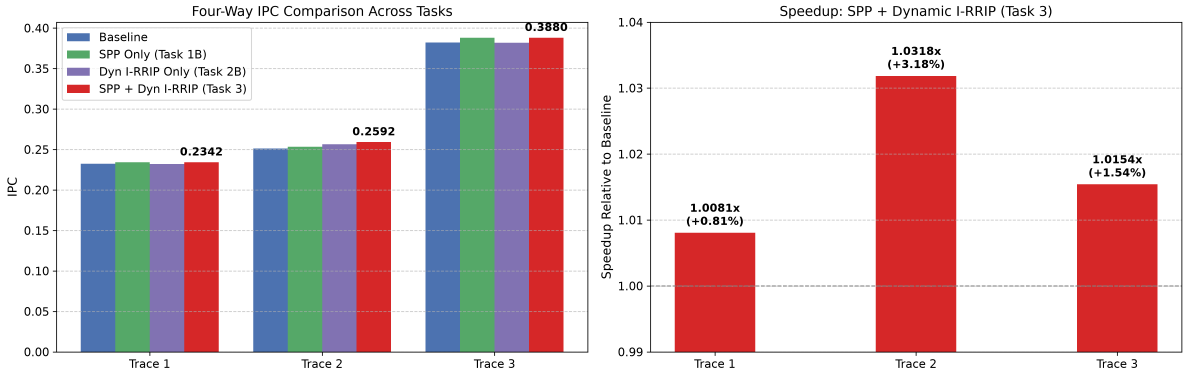


Figure 7: Task 3: Four-Way IPC Comparison and Speedup across all Configurations.

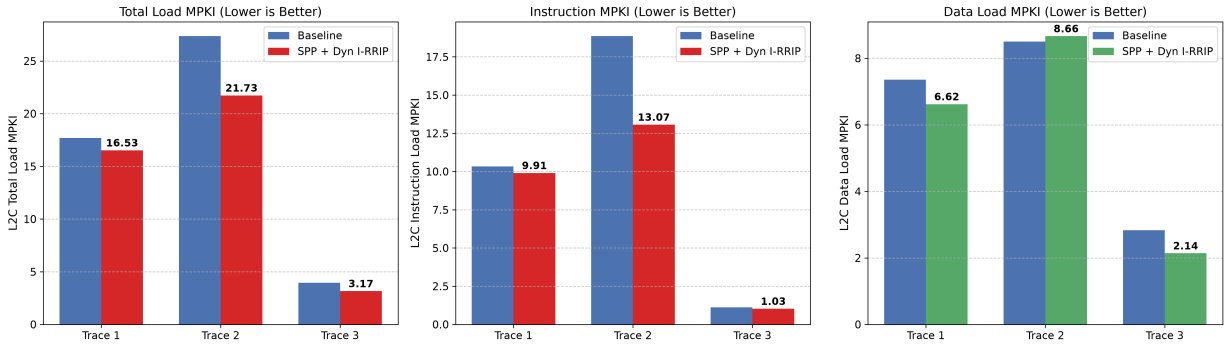


Figure 8: Task 3: L2C Load MPKI Breakdown (Total, Instruction, Data) for Baseline vs Task 3.

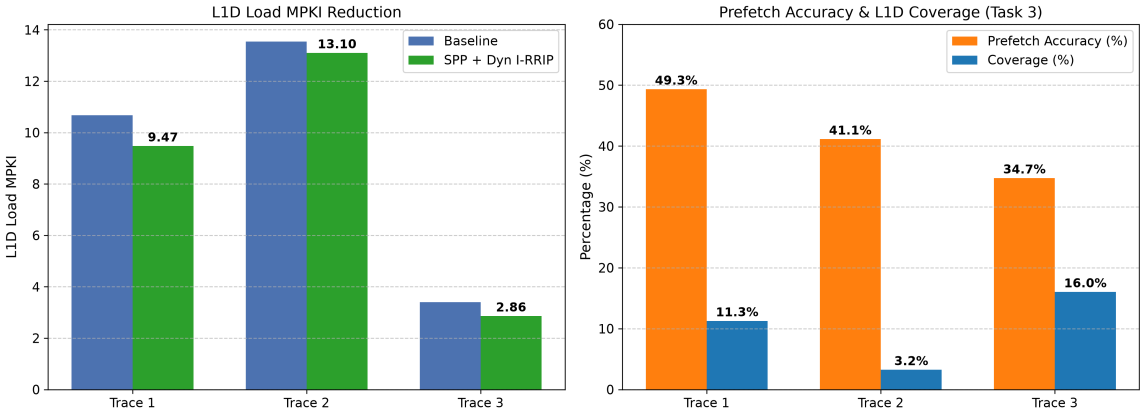


Figure 9: Task 3: L1D Metrics (Load MPKI Reduction, Prefetch Accuracy, and Miss Coverage).

Performance Observations: In Trace 2, combining SPP with Dynamic I-RRIP produces the best result in the entire study:

- Standalone SPP achieves a +0.89% speedup.
- Standalone Dynamic I-RRIP achieves a +2.09% speedup.
- **Combining both techniques in Task 3 delivers a** +3.18% **speedup** (IPC increases from 0.251158 to 0.259154).
- L2 total load MPKI drops from 27.36 down to 21.73, saving **140,616 L2 misses** compared to the baseline.